

Estudi previ P-PRE2

BLE, provisioning de la Wi-Fi i API REST

Pau Vila Perez

Bloc A • BLE

A1. Diferència entre GAP i GATT

El GAP és la part del BLE que s'encarrega que el dispositiu es vegi i que t'hi puguis connectar. És el que fa que el node s'anuncii i que el mòbil el trobi quan busca dispositius. També defineix els rols, per exemple perifèric (el node) i central (el mòbil).

El GATT és el que ve després, quan ja estàs connectat. Organitza les dades que té el dispositiu i diu com es poden llegir, escriure o rebre com a notificació.

Per tant, el que fa que el dispositiu sigui visible és el GAP, i el que fa que se'n puguin llegir dades és el GATT.

A2. Pot ser servidor i client GATT alhora?

Sí. Al principi pensàvem que el perifèric sempre havia de ser el servidor, però són dues coses diferents. El GAP diu qui s'anuncia i qui es connecta, i el GATT diu qui té les dades (servidor) i qui les demana (client). Un cop connectats, un mateix aparell pot fer les dues coses.

Un exemple és un rellotge intel·ligent: fa de servidor quan el mòbil li llegeix les pulsacions, i fa de client quan ell demana l'hora o les notificacions al mòbil.

A3. Jerarquia servei → característica → descriptor

Ho entenem com tres nivells, un dins de l'altre. El servei és un grup de dades que van juntes, la característica és cada dada concreta, i el descriptor és informació extra sobre una característica. Amb el nostre node quedaria així:

- Servei "Ambient"
 - Característica "Temperatura": es pot llegir i també avisa (notifica) quan canvia.
 - Descriptor: serveix per activar les notificacions, i un altre diu que el valor va en graus.
 - Característica "Consigna": el mòbil hi escriu la temperatura que vol.
 - Descriptor: un text que explica què és i en quina unitat va.

Seria com una botiga: el servei és la secció, la característica és el producte i el descriptor és l'etiqueta.

A4. UUID de 16 bits i de 128 bits

Cada servei i cada característica té un UUID, que és un codi que la identifica. Ens va costar entendre que no és el nom del dispositiu: el nom és només perquè el vegin les persones.

- Els de 16 bits són curts i ja estan reservats pel Bluetooth SIG per a coses estàndard, com el nivell de bateria o el ritme cardíac. Com que són curts ocupen poc, i això va bé perquè el paquet d'anunci només té 31 bytes.
- Els de 128 bits són llargs i es fan servir quan el servei és teu i no existeix a l'estàndard. Es generen aleatòriament, i així no coincideixen amb els de ningú altre.

Al nostre projecte farem servir UUID de 128 bits per als serveis propis (el de configurar la Wi-Fi i el de temperatura). Els de 16 bits només si fem servir alguna cosa estàndard.

A5. MTU i contrasenya de 40 caràcters

La MTU per defecte del BLE és de 23 bytes. D'aquests, 3 són de capçalera, així que en una notificació només queden 20 bytes útils.

Una contrasenya de 40 caràcters no hi cap. Hi ha dues solucions:

1. Demanar una MTU més gran quan es connecten. Es pot pujar, per exemple, a 247 bytes, i llavors la contrasenya cap en un sol missatge.
2. Partir la contrasenya en trossos de 20 bytes, enviar-los un darrere l'altre, i que el node la torni a ajuntar.

Bloc B • Provisioning

B1. Tres maneres de passar la Wi-Fi al node

1. SoftAP: el node crea la seva pròpia xarxa Wi-Fi, t'hi connectes amb el mòbil i poses el nom i la contrasenya en una pàgina web. Problema: has de sortir de la teva Wi-Fi i durant una estona et quedes sense internet. És una mica liós per a l'usuari.
2. BLE: el mòbil es connecta al node per Bluetooth amb una app i li envia les dades. Problema: cal una app i el node ha de portar el Bluetooth, que ocupa memòria.
3. SmartConfig: el mòbil envia la contrasenya amagada dins dels paquets Wi-Fi i el node els escolta. Problema: no sempre funciona perquè depèn del router, i és poc segur.

B2. SoftAP vs BLE

Experiència d'usuari: amb SoftAP l'usuari ha de canviar de xarxa, obrir una web i després tornar a la seva Wi-Fi. Amb BLE no ha de canviar res: obre l'app i ja està. Guanya el BLE.

Seguretat: la xarxa del SoftAP sovint està oberta i qualsevol que estigui a prop la pot veure. El BLE té menys abast i les dades poden anar xifrades. Guanya el BLE.

Memòria: el SoftAP aprofita la Wi-Fi, que el node ja necessita igualment. El BLE vol dir afegir tota la part de Bluetooth, que ocupa força memòria, i a més comparteix la ràdio de 2,4 GHz amb la Wi-Fi. Guanya el SoftAP.

Tot i això, la majoria de productes que comprem fan servir BLE, perquè per a l'usuari és molt més còmode.

B3. Què és la NVS

La NVS és una part de la memòria flash de l'ESP32 on es poden guardar dades que no s'esborren quan s'apaga. Funciona amb un nom i un valor, per exemple "ssid" = "WifiCasa". Allà és on guardarem la Wi-Fi.

La diferència amb una variable global és que la variable està a la RAM, i quan apagues o reinicies l'ESP32 es perd. El que hi ha a la NVS es queda.

Quan tornem a carregar el programa (reflashear), la NVS no s'esborra, perquè només es reescriu la part del programa. Les dades només es perden si esborrem tota la flash expressament.

B4. Màquina d'estats del node

Els estats per on passa el node des que s'engega fins que publica són:

1. Arrencada: s'engega i mira si a la NVS hi ha una Wi-Fi guardada.
2. Configuració per BLE: si no n'hi ha, s'anuncia per Bluetooth i espera que el mòbil li envii el nom i la contrasenya.
3. Connectant a la Wi-Fi: quan ja té les dades, apaga el BLE i intenta connectar-se. Si falla, ho torna a provar.
4. Publicant dades: ja està connectat i envia les dades. Si perd la Wi-Fi, torna a l'estat anterior.

Cal tenir en compte que l'ESP32 no pot fer anar bé el BLE i la Wi-Fi alhora, perquè comparteixen la ràdio. Per això, quan ja té la Wi-Fi, apaga el BLE i deixa d'anunciar-se.

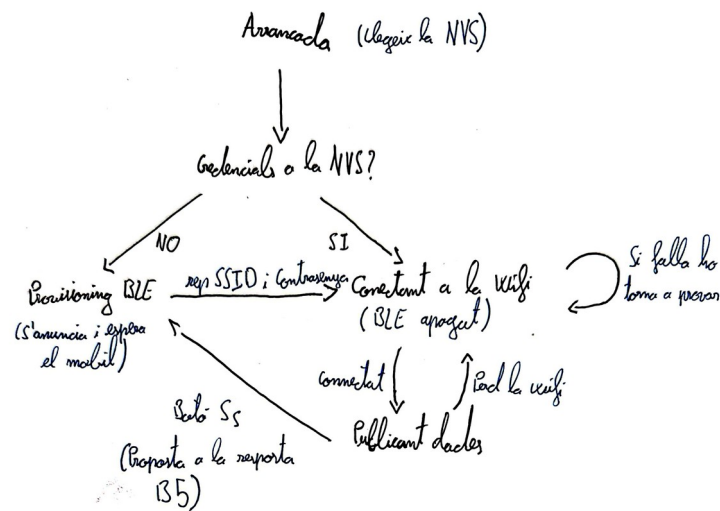


Figura 1. Màquina d'estats del node, dibuixada a mà.

B5. Què passa si l'usuari canvia de router

Si canvies de router, el node continua tenint guardada la Wi-Fi antiga i es quedaria intentant connectar-se tota l'estona. La nostra proposta és:

- **Un botó:** si mantens premut el botó de la placa (a l'AtomS3 és la mateixa pantalla, que es pot prémer) durant 5 segons, el node esborra la Wi-Fi de la NVS i torna a l'estat de configuració per BLE. Un LED fa pampallugues perquè l'usuari sàpiga que ha funcionat. Això ho pot fer qualsevol persona.
- **Automàtic:** si el node falla molts cops seguits en connectar-se, torna sol a la configuració per BLE, però sense esborrar la Wi-Fi guardada, per si només era que el router estava apagat una estona.

Bloc C • REST

C1. Què vol dir idempotent

Una operació és **idempotent** si fer-la una vegada o deu vegades deixa el servidor igual.

- **GET:** sí que és idempotent. Només llegeix, no canvia res.
- **PUT:** sí. Posa un valor concret; si poses la consigna a 22 dues vegades, continua a 22.
- **DELETE:** sí. Si esborres una cosa i ho tornes a fer, continua esborrada.
- **POST:** no. Cada vegada que el fas pot crear una cosa nova, per exemple un registre nou.

C2. Codis d'estat

- **Consulta correcta:** 200 OK.
- **Consigna fora de rang:** 400 Bad Request, perquè l'error és de qui envia la petició (el valor no és vàlid). Si el JSON està ben escrit però el valor és fora de rang, encara és més precís el 422 Unprocessable Content.
- **Node encara no configurat:** 503 Service Unavailable, perquè el node encara no està llest però ho estarà més endavant. També es podria fer servir el 409 Conflict.

C3. Per què REST no va bé per a telemetria amb bateria

Per enviar dades tota l'estona no va bé, perquè cada vegada has d'obrir una connexió, enviar molta informació extra (les capçaleres HTTP) per una dada molt petita, esperar la resposta i tancar. Tot aquest temps la Wi-Fi està encesa, i al M5 vam veure que la ràdio és el que més gasta. Amb bateria, s'acabaria molt ràpid. Per a això és millor MQTT, que manté la connexió oberta i envia molt poca cosa extra.

En canvi, per configurar el node o mirar com està sí que és raonable, perquè només es fa de tant en tant. Aquí no importa gastar una mica més, i és molt còmode: amb un GET veus l'estat i amb un PUT canvies la consigna, i fins i tot ho pots provar des del navegador.

Preparació pràctica

Captura de l'arbre GATT (exemple BLE_server)

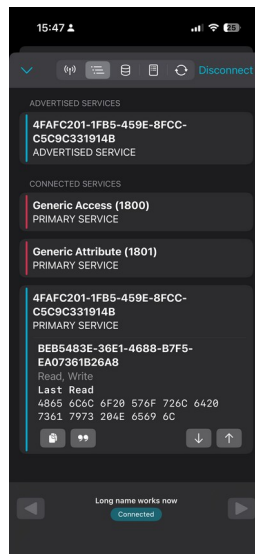


Figura 2. Arbre GATT vist des de l'iPhone amb nRF Connect: el servei propi 4FAFC201... i la seva característica BEB5483E... (Read, Write). El valor llegit, en hexadecimal, és el text "Hello World says Neil".

Adreça BLE del node: AC:27:6E:D1:9D:61 **Tipus:** pública

Com que l'iPhone no mostra l'adreça, hem fet un petit programa que la treu pel port sèrie i l'hem comparat amb l'adreça Bluetooth gravada de fàbrica al xip. Són iguals i no canvia en reiniciar la placa, així que el node fa servir l'adreça pública.

```
Adreça BLE del node:    ac:27:6e:d1:9d:61
Adreça BT de fabrica:   ac:27:6e:d1:9d:61
-----
Adreça BLE del node:    ac:27:6e:d1:9d:61
Adreça BT de fabrica:   ac:27:6e:d1:9d:61
-----
```

Escaneig de xarxes Wi-Fi (SSID, canal i RSSI)

```
Escanejant xarxes...
S'han trobat 11 xarxes:
Nr | SSID                | Canal | RSSI
 1 | iPhone de Oscar     | 6     | -59 dBm
 2 | eduroam              | 1     | -61 dBm
 3 | UVic-UCC            | 1     | -61 dBm
 4 | Lab-Modul           | 1     | -62 dBm
 5 | eduroam              | 5     | -64 dBm
 6 | UVic-UCC            | 5     | -64 dBm
 7 | eduroam              | 9     | -65 dBm
 8 | UVic-UCC            | 9     | -66 dBm
 9 | eduroam              | 13    | -69 dBm
10 | Lab-Modul           | 13    | -70 dBm
11 | Mi 9T Oleguer       | 10    | -79 dBm
```

Escaneig fet el 05/10/2026 des del laboratori. La xarxa del laboratori, Lab-Modul, es veu des de dos punts d'accés (canals 1 i 13). L'ESP32-S3 només veu xarxes de 2,4 GHz.

Node-RED

Comprovat al laboratori el 05/10/2026: sí. Amb el mòbil a la Wi-Fi Lab-Modul, <http://10.11.0.47:1880> obre l'editor del Node-RED. La IP de la Raspberry ha canviat respecte de la P1 (abans 10.11.0.233) perquè la dona el DHCP.